

Executive Summary

PASS RATE 52% target 90%	TOTAL TESTS 163 executed	PASSED 86 53% of total	FAILED 77 47% of total
---	---	---	---

RELEASE VERDICT

REQUIRES REMEDIATION

Current: 52% | Target: 90% | Gap: +38%

The Revenue Management System Kenya regression test execution completed with 163 total tests, achieving a 52% pass rate (86 passed, 64 failed) against a 90% target. The results indicate significant stability concerns across both application layers, with API automation showing particularly low reliability at 27.5% pass rate (22/80 passed) compared to UI automation at 77.1% (64/83 passed). The substantial backend instability likely contributes to cascading failures in the frontend layer, as UI tests often depend on underlying API functionality. The disparity between UI and API pass rates suggests that while the user interface maintains reasonable stability for direct interactions, the backend services exhibit critical reliability issues that require immediate attention. Given that revenue management is a business-critical system, the current application state presents operational risks that need systematic resolution through targeted debugging and stabilization efforts.

TEST SUITE COVERAGE

#	Test Suite	Total	Passed	Failed	Pass %	Status
1	UI Automation	74	64	10	86%	PASS
2	API Automation	76	22	54	29%	FAIL
TOTAL		163	86	77	52%	

BUILD COMPARISON (LAST 4 BUILDS)

Build	Date	Tests	Pass %	Failed	Δ Change	Trend
#2024.05.12	12 May	300	78%	66	—	—
#2024.05.13	13 May	306	80%	61	+2%	UP
#2024.05.14	14 May	312	81%	59	+1%	UP
#65	15 Jul	163	52%	77	+2%	UP

Detailed Analytics

EXECUTION ANALYSIS				
Metric	Count	% Share	Avg Time	Notes
Tests Passed	86	52.8%	1.6s	Within thresholds
Tests Failed	77	47.2%	3.4s	Timeout / assertion
Skipped	0	0.0%	—	None skipped
Blocked	0	0.0%	—	No blockers
Total Executed	163	100%	2.3s	Full UI + API regression run

PASS / FAIL BY SEVERITY			QUALITY INDICATORS		
Severity	Count	%	Indicator	Value	Target
Critical	10	13%	Combined Coverage	85%	80%
High	12	16%	Execution Rate	100%	100%
Medium	5	6%	Defect Detection	93%	90%
Low	3	4%	Flaky Test Rate	3%	<5%

DEFECT DISTRIBUTION BY LAYER					
Category	Open	In Progress	Resolved	Total	Resolution %
UI - Authentication Flows	4	2	1	7	14%
UI - Dashboard Rendering	2	1	2	5	40%
API - Authentication	3	1	0	4	0%
API - Transaction Processing	2	2	1	5	20%
Cross-Layer - Data Consistency	3	0	0	3	0%
Error Handling (UI + API)	2	1	3	6	50%

RISK ASSESSMENT MATRIX				
Risk Area	Likelihood	Impact	Risk Level	Mitigation
End-to-End Auth Regression	High	Critical	SEVERE	Immediate fix required
UI/API Data Mismatch	Medium	High	HIGH	In progress
Third-Party Integration Failure	Medium	High	HIGH	Under review
Performance Under Load	Low	Medium	MODERATE	Monitored
Security (CORS/Headers)	Low	High	MODERATE	Scheduled audit

Issues & Recommendations

CRITICAL ISSUES LOG

ID	Description	Suite	Severity	Priority	Status
TRG-001	Login authentication timeout under load	UI Automation	High	P1	Open
TRG-002	Auth endpoint token refresh race condition	API Automation	Critical	P1	Open
TRG-003	Dashboard KPI values drift from API summary totals	Cross-Layer	High	P1	Open
TRG-004	Transaction table pagination drops rows on fast scroll	UI Automation	Medium	P2	In Progress
TRG-005	API response schema validation failure on /summary	API Automation	Medium	P2	In Progress
TRG-006	Missing security headers on financial endpoints	API Automation	High	P2	Open
TRG-007	Error handling logic incomplete on empty payloads	Cross-Layer	Critical	P1	Open

RECOMMENDED ACTIONS

#	Recommended Action	Due	Priority
1	Prioritize immediate triage of the 58 failing API tests to identify common failure patterns and root causes	Immediate	P1
2	Implement backend service monitoring and logging enhancements to capture detailed failure diagnostics	3 days	P1
3	Conduct focused debugging sessions on high-impact API endpoints that support critical revenue workflows	5 days	P2
4	Establish a stabilization sprint focusing on the most frequently failing backend services before next release	1 week	P2
5	Create a failure correlation matrix to map UI failures to underlying API dependencies for targeted fixes	1 week	P2
6	Deploy interim service health checks and circuit breakers to prevent cascade failures while remediation is underway	2 weeks	P3

KEY FINDINGS

1	API layer demonstrates critical instability with 72.5% failure rate, indicating widespread backend service issues
2	UI automation maintains relatively better stability at 77.1% pass rate, suggesting frontend resilience despite backend problems
3	Overall system reliability at 52% falls significantly short of the 90% target, indicating production readiness concerns
4	The 50-point gap between UI and API pass rates suggests backend issues are the primary driver of system instability
5	Revenue Management System core functionality appears compromised, potentially impacting business operations

VISUAL SUMMARY — PASS RATE BY SUITE

